
DESARROLLO DE UN SISTEMA DE GESTIÓN DE RECURSOS, CONFIGURACIONES E INSTANCIAS PARA TECNOLOGÍAS CHAPINAS S.A. CON INTEGRACION XML Y FACTURACIÓN AUTOMATIZADA

202400111 – Richard Steven Arizandietz Yol

Resumen

El presente proyecto desarrolla un sistema informático integral para la empresa ficticia Tecnologías Chapinas S.A., orientado a la gestión de recursos tecnológicos, configuraciones e instancias mediante una arquitectura web basada en Django y Flask.

La solución permite la carga masiva de datos en formato XML, su validación y almacenamiento estructurado en JSON, así como la generación automática de facturas en PDF y XML. Este enfoque promueve la interoperabilidad y el uso eficiente de recursos, alineándose con tendencias actuales de automatización y gobierno de datos en entornos empresariales.

Entre los impactos más relevantes se destacan la reducción de errores humanos en la facturación, la optimización del tiempo de configuración y el fortalecimiento de la trazabilidad técnica.

Palabras clave

Gestión de recursos, XML, Django, Flask, Facturación automatizada.

Abstract

This project presents the development of an integrated information system for the fictional company Tecnologías Chapinas S.A., designed to manage technological resources, configurations, and instances through a web architecture built with Django and Flask.

The solution supports massive data loading in XML format, including validation and structured storage in JSON, as well as automated invoice generation in both PDF and XML. This approach promotes interoperability and efficient resource management, aligning with current trends in automation and data governance within business environments.

Among the main impacts are the reduction of human errors in billing, optimization of configuration times, and reinforcement of technical traceability.

Keywords

Resource management, XML, Django, Flask, Automated billing

Introducción

El avance de la transformación digital ha impulsado a las empresas a modernizar sus procesos mediante soluciones tecnológicas que integren eficiencia, control y automatización. En este contexto, el proyecto Tecnologías Chapinas S.A. propone el desarrollo de una plataforma web que facilita la gestión de recursos, configuraciones e instancias virtuales, optimizando la administración de servicios tecnológicos y su facturación asociada. La propuesta utiliza una arquitectura distribuida que combina Django en el front-end y Flask en el back-end, comunicándose a través de servicios REST, lo cual permite una mayor flexibilidad, escalabilidad y modularidad del sistema. Asimismo, el uso de archivos XML y JSON garantiza la interoperabilidad con otros sistemas y la trazabilidad de los datos. Este trabajo se enmarca en las necesidades actuales de automatización empresarial, aportando una solución eficiente y adaptable para pequeñas y medianas empresas que buscan fortalecer su gestión tecnológica con herramientas de código abierto y bajo costo.

Desarrollo del proyecto

Objetivos del Sistema:

El sistema tiene como objetivo principal automatizar la gestión de recursos tecnológicos ofrecidos por Tecnologías Chapinas S.A., permitiendo a los clientes crear configuraciones personalizadas, instancias de servicio y generar facturación automática según el uso de recursos. De manera específica, busca:

- Centralizar la información de recursos, configuraciones, clientes e instancias.
- Permitir el registro y consulta mediante una interfaz web intuitiva.

- Calcular costos de operación basados en horas de uso y valor por recurso.
- Generar facturas en formato PDF y XML, promoviendo la trazabilidad contable. Garantizar la interoperabilidad a través del uso de estructuras JSON y XML.

Arquitectura del Sistema:

El proyecto adopta una arquitectura de tres capas, compuesta por:

- Front-end (Django): Gestiona la interacción con el usuario, mostrando formularios, listados y reportes.
- Back-end (Flask): Expone una API REST que procesa las solicitudes del cliente y gestiona la lógica de negocio.
- Capa de datos: Implementada mediante archivos JSON persistentes, complementados por XML para la importación y exportación masiva.

La comunicación entre Django y Flask se realiza mediante peticiones HTTP, garantizando independencia entre los componentes y facilitando su despliegue modular en diferentes entornos.

Clases principales:

Las clases fundamentales del sistema se agrupan en torno a las entidades del dominio:

- Cliente: almacena los datos del usuario y su identificación dentro del sistema.
- Recurso: define los elementos tecnológicos ofrecidos (CPU, RAM, almacenamiento, licencias).
- Configuración: representa un conjunto de recursos predefinidos, clasificados por categoría. Instancia: asocia a un cliente con una configuración activa, incluyendo fechas, estado, y costos.
- Factura: compila el consumo por cliente y genera los documentos XML y PDF correspondientes.

Flujos de ejecución

El flujo principal inicia con la creación de configuraciones y la definición de recursos base. Los clientes eligen una configuración, lo que genera una instancia que se activa con una fecha de inicio y permanece vigente hasta su cancelación. Durante este proceso, el sistema calcula automáticamente el costo total multiplicando el valor por hora de cada recurso por las horas de uso registradas. Finalmente, el módulo de facturación recorre las instancias vigentes o canceladas, genera los totales acumulados por cliente y produce los reportes finales en ambos formatos (XML y PDF).

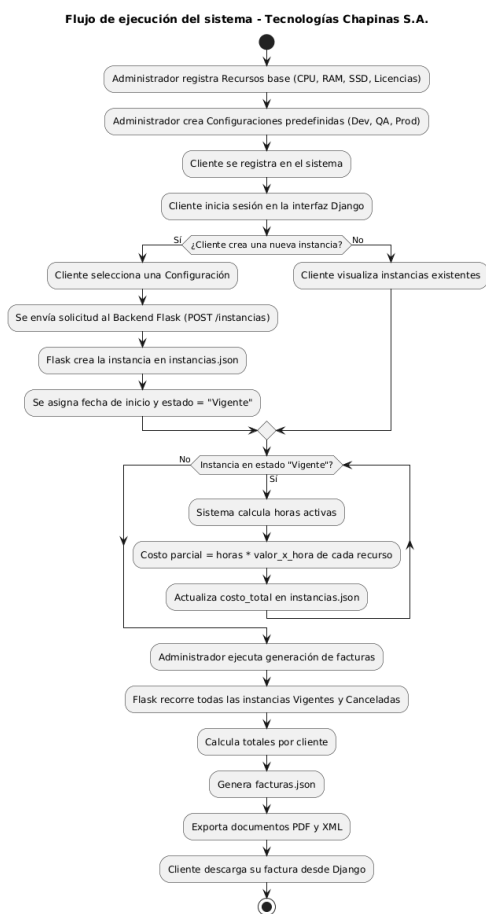


Figura 1. Flujo de ejecución del sistema.

Fuente: elaboración propia.

Estructuras de datos

La persistencia de la información se maneja mediante archivos JSON, organizados en entidades independientes (clientes.json, recursos.json, configuraciones.json, instancias.json, facturas.json). El formato XML se utiliza para intercambiar datos masivos y definir la estructura jerárquica de los objetos, incluyendo esquemas XSD que validan la integridad de los documentos. Esta combinación ofrece un equilibrio entre simplicidad, portabilidad y compatibilidad con tecnologías modernas de integración.

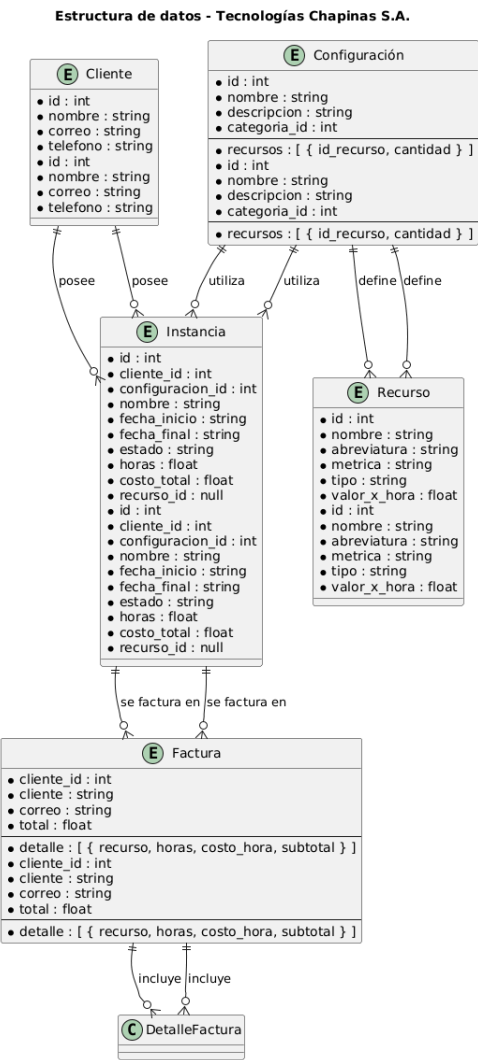


Figura 2. Estructura de datos del sistema.

Fuente: elaboración propia.

Despliegue de componentes

El sistema puede desplegarse en entornos locales o en servidores distribuidos. El back-end Flask se ejecuta como servicio API independiente, mientras que el front-end Django se aloja en un servidor web (como Unicorn o Nginx). La comunicación entre ambos se realiza mediante rutas REST (/api/recursos, /api/configuraciones, /api/instancias, /api/facturas). La generación de documentos PDF y XML se efectúa en el servidor de Flask, permitiendo su descarga directa desde el cliente. Este diseño modular facilita futuras expansiones hacia servicios en la nube o bases de datos relacionales, sin alterar la lógica principal.

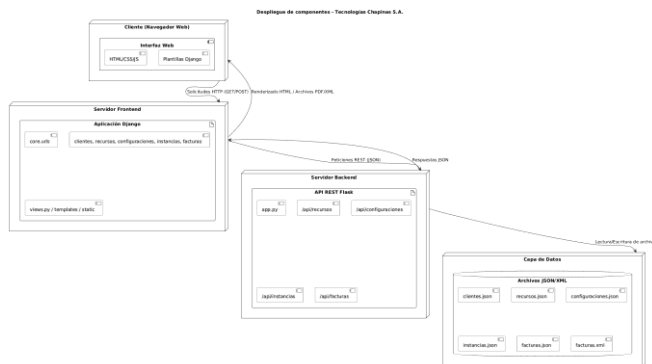


Figura 3. Despliegue de componentes del sistema.

Fuente: elaboración propia.

Reportes Generados

El sistema desarrollado para Tecnologías Chapinas S.A. incorpora la capacidad de generar reportes automáticos tanto en formato PDF como en XML, los cuales reflejan la información de consumo, costos e instancias activas de cada cliente. Estos reportes se producen desde la capa backend (Flask) y pueden descargarse directamente desde la interfaz frontend (Django).

1. Reporte de Facturas (PDF)

- Ruta: /api/facturas/pdf
- Formato: Documento PDF legible y exportable.
- Contenido:

- Nombre y correo del cliente.
- Recursos utilizados por cada instancia.
- Costo por hora, horas utilizadas y subtotal por recurso.
- Total general facturado por cliente.

- Propósito: Facilitar la presentación de facturación en un formato visual e imprimible, adecuado para entrega al cliente o respaldo administrativo.
- Generación: Se usa la librería reportlab para construir dinámicamente el PDF, asegurando paginación, encabezados y separación por clientes.

2. Reporte de Facturas (XML)

- Ruta: /api/facturas/xml
- Formato: Documento estructurado en XML, basado en la jerarquía:

```
<facturas>
  <factura>
    <cliente>Juan Pérez</cliente>
    <correo>juan@example.com</correo>
    <total>125.5</total>
    <detalle>
      <item>
        <recurso>CPU Virtual</recurso>
        <horas>10</horas>
        <costo_hora>1.2</costo_hora>
        <subtotal>12.0</subtotal>
      </item>
    </detalle>
  </factura>
</facturas>
```

- Propósito: Permitir la interoperabilidad con otros sistemas, ya que el formato XML puede ser procesado por plataformas contables o integraciones de terceros.
- Generación: Se emplea xml.etree.ElementTree, garantizando una estructura conforme a las buenas prácticas de datos jerárquicos.

3. Reporte de Consumos (JSON Interno)

- Ruta: /api/facturar
Formato: Archivo JSON (facturas.json) con toda la información consolidada por cliente.
- Uso:
 - Fuente de datos para generar los reportes PDF y XML.
 - Permite recalcular costos sin pérdida de precisión.
 - Registro histórico de consumos por periodo de facturación.



Figura 4. Flujo de generación de reportes.

Fuente: elaboración propia.

Todas las figuras deben ir enumeradas al pie de la imagen, como se muestra en el ejemplo.

En el caso de inclusión de tablas, éstas deben pegarse en el formato de origen, conservando el modelo mostrado en el cual pueden agregarse las columnas o filas que sean necesarias.

Conclusiones

El desarrollo del sistema para la administración de recursos y facturación en la nube de Tecnologías Chapinas S.A. permitió consolidar una arquitectura moderna y modular, que integra tecnologías web ampliamente utilizadas como Django y Flask. Esta integración favorece la separación entre la capa de presentación y la lógica de negocio, garantizando una comunicación eficiente mediante servicios RESTful.

El proyecto evidencia cómo una correcta gestión de configuraciones, instancias y recursos puede optimizar procesos de facturación y control operativo en entornos tecnológicos. Desde el punto de vista técnico, la implementación de estructuras de datos en formato JSON y reportes PDF/XML brinda flexibilidad, interoperabilidad y transparencia en el manejo de la información.

Además, la experiencia obtenida demuestra la importancia de diseñar sistemas escalables y mantenibles, donde la automatización de reportes y la validación de recursos aseguran coherencia en los costos y trazabilidad en el consumo.

Como reflexión final, se sugiere explorar la integración futura de bases de datos relacionales y servicios cloud computing reales (como AWS o Azure), permitiendo a la empresa simular un entorno de producción más cercano a las necesidades del mercado actual. Esta evolución impulsaría no solo la precisión técnica del sistema, sino también su aplicabilidad comercial y profesional en escenarios reales.

Referencias bibliográficas

C. J. Date, (1991). An Introduction to Database Systems. Addison-Wesley Publishing Company, Inc.

Django Software Foundation. (2024). Django Documentation, Version 5.0. Disponible en:
<https://docs.djangoproject.com/>

Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. O'Reilly Media.

McLaughlin, B. (2006). Java and XML, 3rd Edition. O'Reilly Media.

Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.